

# DPF-PIR: A Scheme of Feasible Two-Server Keyword PIR with Logarithmic Communication

Chao Li, Ziyuan Liang, Fan Zhang, Jian Long, Bingsheng Zhang, Jian Liu

**Abstract**—Private Information Retrieval (PIR) allows a client to privately request a block of data from a database such that no information about the queried block is revealed to the database owner. With the rapid rise of cloud computing, data is often shared across multiple servers, making multi-server PIR a promising privacy-enhancing technology. As the demand for faster keyword PIR protocols increases, current single-server PIR schemes suffer from significant computational and communication costs, while two-server PIR schemes demonstrate superior performance in this regard. In this paper, we address the problem of the keyword PIR against some adversary who can corrupt at most one party in our protocols in the semi-honest setting. A feasible two-server scheme *DPF-PIR* is presented, inspired by the original employment of the distributed point function [1]. Without the need of downloading some “hint” about the database, *DPF-PIR* can achieve similar throughput results with the state-of-the-art single-server scheme, SimplePIR [2], and  $25.5\times$  faster than previous schemes. Meanwhile, the communication cost of *DPF-PIR*, which exhibits *logarithmic complexity*, is significantly lower compared to other schemes; for example, it is less than 2% of the communication cost of SimplePIR. We also present a variant of our scheme, *PDPF-PIR*, rendering the non-collusion assumption more acceptable in practice. Despite the decreased throughput due to heavier computational costs, *PDPF-PIR* is still at least  $2\times$  faster than previous single-server schemes.

**Index Terms**—Private Information Retrieval, Distributed Point Function, Secure Multi-party Computation

## I. INTRODUCTION

In today’s digital landscape, there exists an elevated demand for securely accessing files from the Internet. Nonetheless, content providers face a substantial challenge in guaranteeing the security of user requests and shielding against potential attacks, thereby impeding the establishment of trust. A solution to the trust problem is private information retrieval (PIR). In a PIR protocol, a database server holds an array of  $N$  records. A client wants to fetch record  $i \in \{1, \dots, N\}$  from the server, without revealing the desired index  $i$  to the server. There is a multitude of applications for PIR schemes, with different reasons for hiding the identity of the item requested by the user. A typical motivation is to prevent leakage of personal or business interest in specific information from a collection, e.g., patents, medical articles, company evaluations, product descriptions. For example, voice call metadata—the parties involved in the call, the duration of the call, and the time of the call—leaks a lot of information about the content of the call [3]. PIR protocols can be utilized to unlink the callee from the caller in this scenario. Despite the application potential and the great advances in PIR protocols, there have been essentially no large-scale deployments of PIR technology to date.

Recent PIR schemes [2], [4], [5] exhibit a polylogarithmic growth in the total communication required to fetch a database record as the number of records,  $N$ , increases, which is acceptable in practice. However, PIR schemes typically use public-key cryptography or homomorphic encryption to prevent the server providing the service from obtaining information related to the query. The server must touch every bit of the database to answer even a single client query. Otherwise, the PIR scheme disclose which database record the client is not interested in. Therefore, the performance issue of cryptography primitives is further amplified. The performance of most existing PIR schemes falls short of meeting practical application requirements of throughput, especially in streaming scenario. This study aims to enhance the practical applicability of PIR, as distributed point function (DPF) technique can facilitate its implementation.

In this paper, we present two new keyword PIR schemes that both the computation burden and communication costs are low under two-server setting. More specifically, our first scheme, *DPF-PIR*, achieves a server throughput of 5.5GB/s/core, or 90% of the theory uplimit, XOR PIR [6], while the communication costs of XOR PIR can be  $450\times$  more than *DPF-PIR*. Our second scheme, *PDPF-PIR*, extends the settings of the scheme, supporting online/offline differentiation, which means that only the online server needs to be queried during the online phase. *PDPF-PIR* achieves a lower server throughput of 418MB/s/core, but shrinks the communication cost to roughly half the *DPF-PIR* needs.

### A. Our Contributions

In summary, our contributions are listed as below.

- We propose a new two-server keyword PIR scheme, *DPF-PIR*, based on the DPF primitive, which provides the logarithmic communication costs and low computation burden.
- We propose a variant of *DPF-PIR*, called *PDPF-PIR*, to extend the two-server setting to the online/offline server setting. The variant presents the similar performance with *DPF-PIR*.
- We provide the simulation-based proof over the security of the two schemes under non-collusion assumption against a semi-honest adversary who can corrupt at most one party in the schemes, demonstrating the privacy preservation. The implementation and evaluation of our proposed schemes are also presented, which demonstrate that our schemes offer a considerable advantage in terms of throughput and communication costs,  $113\times$  and  $2\%$

respectively, compared with some representative PIR schemes.

### B. Organization

Sec. II shows some preliminary knowledge. Sec. III presents our construction details. Sec. IV explores the security of the proposed schemes, including security assumptions and simulation-based proofs. Sec. V presents a comprehensive explanation of the implementation of the proposed schemes, followed by an evaluation against state-of-the-art PIR schemes in Sec. VI. Finally, Sec. VII concludes this paper.

## II. PRELIMINARIES

In this section, we provide preliminaries on KPIR and associated security assumptions and introduce the primitives and algorithms employed in our work.

### A. Semi-honest Security

Following Goldreich's definition [7], protocols that are secure in the presence of semi-honest (or honest-but-curious) adversaries assume that all parties faithfully follow all protocol specifications and do not misrepresent any information related to their input, e.g., DB size and content. However, parties may attempt to collect additional information during the protocol execution to infer other parties' private data. In contrast, malicious security requires that adversaries try to break the security of the protocol in any possible ways, such as stealing private data or sending malicious messages, while the protocol shall still ensure data privacy and integrity.

### B. Keyword Private Information Retrieval

PIR [6] is a cryptographic protocol that allows a user to retrieve an item from a server in possession of a database without revealing which item is retrieved. The main goal of PIR is to provide privacy to the user while retrieving information from a database.

In general, there are two ways to address the problem of PIR. One way is to make the server computationally bounded, i.e., computational PIR [8]. The other way is to assume that there are multiple non-cooperating servers, each with a copy of the database, which is known as the information-theoretic PIR [6]. A trivial but inefficient approach to achieve PIR is to drive the server to send an entire copy of the database to the user. Accordingly, more PIR protocols are developed to perform tasks more efficiently, including the keyword PIR.

Typically, elements are retrieved using the physical address of the desired item, which we call index PIR to distinguish it from another variant called keyword PIR (KPIR). In KPIR, the user's desired element is retrieved using an identifier connected to the desired item, e.g., the name of a file. KPIR is useful for the application of private file retrieval [9], which can serve as a database query task. The ideal functionality is given in Algorithm 1. In this paper, we focus on the two-server keyword PIR against semi-honest adversaries.

**Semi-honest Security:** Let  $\text{view}_1^\pi(x, \text{DB}, \lambda)$ ,  $\text{view}_2^\pi(x, \text{DB}, \lambda)$  and  $\text{view}_3^\pi(x, \text{DB}, \lambda)$  be the views of the client and two servers

---

### Algorithm 1: Keyword PIR ideal functionality $\mathcal{F}_{\text{KPIR}}$

---

PARAMETERS: A client  $\mathcal{C}$ , and a database DB with size  $N$ , item length  $\rho$ .

FUNCTIONALITY:

- Wait for input query keyword  $x \in \{0, 1\}^*$
  - Wait for input database  $\text{DB}\{(k_1, v_1), \dots, (k_N, v_N)\} \subset \{(\{0, 1\}^\rho, \{0, 1\}^\rho)\}^*$
  - Give output  $(k, v)$  to  $\mathcal{C}$  satisfying  $(k, v) \in \{(k_1, v_1), \dots, (k_N, v_N)\}$  and  $k = x$
- 

in the protocol  $\pi$ , respectively. Let  $(k, v)$  be the output in the ideal functionality. The protocol  $\pi$  is semi-honest secure if there exist probabilistic polynomial time algorithms  $\mathcal{S}_1, \mathcal{S}_2$  and  $\mathcal{S}_3$  such that for every  $\lambda, x \in \{0, 1\}^*$  and every DB,

$$\mathcal{S}_1(x, (k, v)) \stackrel{c}{\approx} \text{view}_1^\pi(x, \text{DB}) \quad (1)$$

$$\mathcal{S}_2(\text{DB}) \stackrel{c}{\approx} \text{view}_2^\pi(x, \text{DB}) \quad (2)$$

$$\mathcal{S}_3(\text{DB}) \stackrel{c}{\approx} \text{view}_3^\pi(x, \text{DB}) \quad (3)$$

### C. Distributed Point Function

A PIR protocol can be implemented by utilizing distributed point function (DPF) [1], [10]–[12], which is proposed to securely search and update distributed data. A point function  $\hat{f}$  is described by two parameters  $\alpha$  and  $\beta$  as

$$\hat{f}_{\alpha, \beta}(x) = \begin{cases} 0, & x \neq \alpha, \\ \beta, & x = \alpha. \end{cases} \quad (4)$$

The DPF enables the evaluation of point functions to be performed in a privacy-preserving distributed environment. A DPF consists of the following two basic algorithms: *GEN* and *EVAL*, and the functionality is given in Algorithm 2.

---

### Algorithm 2: Distributed Point Function

---

PARAMETERS: Security parameter  $\lambda$ , the input domain  $\{0, 1\}^n$  and output domain  $\mathbb{G}$  of the point function  $f$ .

FUNCTIONALITY:

GEN:  $\text{GEN}(\hat{f}_{\alpha, \beta}) \rightarrow (k_0, k_1)$

- Wait for input point function  $\hat{f}_{\alpha, \beta}$  ( $\alpha \in \{0, 1\}^n$  and  $\beta \in \mathbb{G}$ )
- Output evaluation keys  $(k_0, k_1)$  with key size  $n(\lambda + 2) + \log_2 |\mathbb{G}|$

EVAL:  $\text{EVAL}(k_i, x) \rightarrow y_i$

- Wait for input evaluation key  $k_i$  and evaluation point  $x \in \{0, 1\}^n$
  - Output secret share of function evaluation  $y_i \in \mathbb{G}$
- 

The *GEN* algorithm accepts a point function as input and produces two evaluation keys, denoted by  $k_0$  and  $k_1$ . These keys are subsequently utilized in the *EVAL* algorithm to compute the respective outputs  $y_0$  and  $y_1$ , representing secret shares of the evaluation of the point function. The correctness

property of a DPF is that, if  $(k_0, k_1) \leftarrow \text{GEN}(\alpha, \beta)$ , then  $y_0 \leftarrow \text{EVAL}(k_0, x) \oplus y_1 \leftarrow \text{EVAL}(k_1, x) = 0$  except when  $x$  is equal to  $\alpha$ , in which case the result is  $\beta$ .

DPF divides the evaluation process of a point function into a distributed environment. Privacy-preserving applications using distributed point function as a building block have been widely proposed in varied scenarios, like contact discovery [13], encrypted search system [14], ORAM [15], private-preserving advertising [16], etc. It provides a cryptographic primitive that allows the distributed computation of a point function by dividing its calculation process into two parts, which can be utilized to construct an original two-server PIR scheme. The construction of DPF relies solely on One-Way Function (OWF), which results in low computational overhead. Additionally, the information related to the point function itself is difficult to be stolen with only a single key, making DPF an ideal choice as a core primitive for PIR protocols.

#### D. Programmable DPF

A recent paper [12] presents a new framework for 2-party DPF construction, which is applicable in the setting of feasible (polynomial-size) domains. Their approach is motivated by a strengthened notion of programmable DPF (PDPF), in which a short, input-independent “offline” key can be reused for sharing many point functions. They construct a PDPF for feasible domains from the minimal assumption that one-way functions exist, where the second “online” key size is polylogarithmic in the domain size  $N$ . PDPF differs from DPF in several aspects. One significant difference between PDPF and DPF is the online/offline keys and servers, which is referred as the 1.5-server property in this paper. PDPF offers multiple new efficiency features and applications, such as privately puncturable PRFs [17], [18],  $O(1)$ -round distributed DPF  $\text{GEN}$  and  $O(1)$ -round Doerner-shelat [15] distributed key generation.

The 1.5-server property of PDPF renders it more practical in real-world scenarios. The reason is that the assumption of maintaining two fully synchronized and non-colluding database servers in reality can be challenging. Nevertheless, the 1.5-server property allows the offline server to have a different role than the online server. The offline server gains more trust from clients and can perform pre-computation, which relaxes the security assumptions of PDPF compared to DPF.

The PDPF construction, based on Punctured PRF [19], [20], is similar with the DPF construction, i.e., the result from two servers differs only when  $x = \alpha$ . However, the algorithm based on PPRF results in a higher computational overhead, which is the other side of the high security.

#### E. Cuckoo Hashing

Cuckoo hashing [21], [22] is an open addressing method used to handle collision in hash tables, which features low space occupancy and fast query speed. Cuckoo hashing uses a hash group with  $k$  hash functions  $h_1, h_2, \dots, h_k : \{0, 1\}^* \rightarrow [m]$  to generate the hash table. The hash index of an item is choosed randomly from  $k$  indexes. The classical cuckoo

hashing employs a special storage area (called stash) to store keys that fails to be placed in the main hash table. The cuckoo table is capable of allocating a unique entry for every inserted item with a relatively small number of entries. Inspired by this property, we have incorporated cuckoo hashing in our proposed scheme to simplify the construction and computation.

#### F. Private Set Intersection

Private set intersection (PSI) stands as a widely-used multiparty cryptographic protocol enabling the determination of common elements in private sets among two or more parties while safeguarding information about elements absent in the intersection. The ideal functionality  $\mathcal{F}_{PSI}$  of a PSI between server  $\mathcal{S}$  on input  $S = \{s_1, \dots, s_w\}$  and client  $\mathcal{C}$  on input  $C = \{c_1, \dots, c_v\}$  is defined as follows:

$$\mathcal{F}_{PSI} : (S, C) \rightarrow (\perp, S \cap C) \quad (5)$$

where  $w, v$  are the public input to  $\mathcal{F}_{PSI}$ .

A PSI variant with payloads, i.e., PSI with associated data (PSI+AD) is more relevant with KPIR problem. The server’s input  $S$  is a set of key-value pair  $S = \{(s_1, v_1), \dots, (s_w, v_w)\}$  instead, then the ideal functionality is defined as follows:

$$\mathcal{F}_{PSI+AD} : (S, C) \rightarrow (\perp, \{(k_i, v_i) | k_i \in C, (k_i, v_i) \in S\}) \quad (6)$$

This variant Equation 6 is necessary for our schemes, and its employment is presented in subsection III-A.

### III. CONSTRUCTION

In this section, we present our proposed *keyword PIR* protocols. With two servers and one client, our protocols are capable of privately retrieving an item in two replicated databases held by two servers. Our protocols employ multiple cryptographic primitives and protect themselves from semi-honest adversaries. The best security properties of the protocols are held when these two servers do not collude.

#### A. DPF-PIR Protocol

Our first scheme for the keyword PIR is called DPF-PIR. We assume that the service provider’s database is replicated on two separate servers. To perform a PIR, a client interacts with both servers simultaneously.

DPF can be used to build a keyword PIR protocol [10] in the two-server setting. However, using the keyword  $x$  itself as the point function parameter  $\alpha$  is undesirable in practice. In our protocol, we introduce the cuckoo hashing technique and the PSI protocol to lower the computation and communication costs. Our protocol consists of the following steps, and a full description of our protocol is presented in Fig. 1.

1) *DPF-based PIR*: First, consider an original PIR scheme based on the DPF. The client wants to retrieve the  $i$ -th item in the DB held by two servers, so the client constructs a DPF instance using the point function  $\hat{f}_{i,1}$ . Then both servers execute the full domain evaluation  $\text{EvalAll}$  (the process of  $\text{EVAL}$  on all leaves) [1]. This PIR scheme has additive reconstruction, the same as the underlying DPF: the client’s

**Input:**  $x$  is the client's query keyword, DB is the database held by server A and server B, where  $x \in \{0,1\}^\rho$ ,  $\text{DB} \subseteq \{\{0,1\}^\rho, \{0,1\}^\rho\}^*$ .

**Parameters:**  $N = |\text{DB}|$ .  $p$  is the number of hash functions.  $\lambda$  is the statistical security parameter.

**Protocol:**

1) **Initialization**

- Two servers agree on the same hashing family  $h_1, \dots, h_p : \{0,1\}^* \rightarrow [M]$ , and cuckoo table size  $M = |T|$ , such that inserting  $N$  items into the cuckoo table fails with probability  $\leq 2^{-\lambda}$ .
- The servers proceed to insert the database into the cuckoo table, ensuring that each data entry is placed within the table, occupying exactly one position.

2) **Query**

- The client obtain information regarding  $p$  hash functions  $\{h_i\}_{i \in [p]}$  from the servers, and computes  $p$  candidate indexes  $\{I_i\}_{i \in [p] | I_i = h_i(x)}$ .
- The client selects each index in the candidate set in a random order, and generates evaluation keys  $(k_0^i, k_1^i) \leftarrow \text{GEN}(I_i, 1)$  for each  $i \in [p]$ . Then, the client transmits the keys to their respective servers, sending  $k_0^i$  to server A and  $k_1^i$  to server B.

3) **Evaluation**

- For each evaluation key  $k_b^i (b \in \{0,1\})$ , the servers compute their own secret shares  $s_b^i = \sum_{j=0}^M (\text{EVAL}(k_b^i, j) \cdot T[j])$ .
- The client generates mask code  $m_i$  randomly for each query, i.e.  $I_i$ , then transmits mask codes to server A. Server A executes a shuffle procedure  $\sigma() : [p] \rightarrow [p]$  to mask codes, and outputs its masked secret shares  $m_{\sigma(i)} \oplus s_0^i$  to server B.
- Server B receives secret shares from server A, and constructs masked results  $m_{\sigma(i)} \oplus s_0^i \oplus s_1^i$ .

4) **Secret Retrieval**

- The client then computes the masked versions of the query keyword  $x \oplus m_i$ . The client and server B respectively send  $u = \{x \oplus m_i\}$ ,  $v = \{m_{\sigma(i)} \oplus s_0^i \oplus s_1^i\}$  to the PSI protocol so that the client receives  $z = u \cap v$ . The client outputs  $x, T[x]$ .

Fig. 1: DPF-PIR: a two-server keyword PIR protocol based on the distributed point function

query output  $\text{DB}[i]$  is equal to the XOR of the responses from the two servers.

Suppose that we change the message flow of the original PIR scheme as follows: the client sends a mask  $m$  to server A. Server A computes its DPF *EvalAll* result  $s_0$  and instead of sending it to the client, sends  $s_0 \oplus m$  to server B. Then server B computes its DPF *EvalAll* result  $s_1$  and obtain a masked result  $s_1 \oplus (s_0 \oplus m) = \text{DB}[i] \oplus m$ . Server B does not acquire knowledge of the mask code  $m$ . Despite receiving two *EvalAll* results, the actual result remains hidden from server B.

To retrieve the query result, the client needs to perform a secure protocol with server B. In this case, a private equality test is performed. Therefore, such a change of message flow does not compromise the security of the DPF, and the PIR functionality is fulfilled. Even though we do not provide a reason for this modification at the moment, the necessity of modifying the message flow will become apparent after finishing the construction.

This original protocol is not capable of performing a keyword PIR between a keyword  $x$  and DB, since the client is not able to know which location in DB to query. We could directly use the keyword  $x$  itself to build a DPF query, but it is not a good choice. First, the keywords that the client chooses and the database stores have various lengths. To avoid the leakage of keyword length, the client and servers are forced to pad the

keyword to the length of the longest keyword or even longer. This causes extra storage and computation waste. Second, the distribution space of the keyword is naturally sparse. Even not considering the padding cost discussed before, the computation of the DPF on the keyword itself is not economic.

The performance of DPF depends on the length of the representation of their inputs. When the original input representation is sparse, which is the case of the keyword PIR problem, we can first use a hash function to map the keywords of the input items to identities from a smaller domain with a shorter representation. Given a keyword set of cardinality  $2^M$  with  $\rho$ -bit elements (normally,  $\rho \gg M$ ), the complexity of using the keyword itself to build a DPF query is  $O(2^\rho)$ . By employing some hash technique to generate a compressed representation of  $M + 1$  bits, we achieve a computational complexity of  $O(2^{M+1})$ . This represents an exponential optimization compared to the original approach.

Next, we will address this problem by organizing the database using cuckoo hashing.

2) *Cuckoo Hashing Optimization for Keyword PIR:* The Cuckoo hashing uses an array with  $m$  entries and  $k$  hash functions  $h_1, \dots, h_k : \{0,1\}^* \rightarrow [M]$ . An item  $x$  is stored in one of the locations indexed by  $h_1(x), \dots, h_k(x)$ . Furthermore, only a single item can be assigned to each entry. Typically,  $k$  is quite small (we use  $k = 3$ ). Previous studies [23] provide a

comprehensive analysis on the possibility of a failed insertion in the cuckoo hashing setting, and results show that the failure probability of a cuckoo table without the stash can still be acceptable by selecting appropriate parameters.

We choose cuckoo hashing parameters such that the failure happens with sufficiently low probability, i.e., no stash is required. We give a pseudocode of this no-stash variant in Algorithm 3.

---

**Algorithm 3:** Cuckoo Hashing Insertion without Stash

---

**Input:** Item to insert,  $x$

**Output:** Index in the hash table,  $i$

1. Compute all candidate indexes

$I_1 \leftarrow h_1(x), I_2 \leftarrow h_2(x), \dots, I_p \leftarrow h_p(x);$

2. Sample a candidate index  $i \xleftarrow{\$} \{I_r : r \in [p]\};$

3. If  $i$  is occupied, evict item  $y$  which occupies index  $i$ , and insert  $x$  into  $i$ ;

4. Re-insert  $y$ ;

5. Repeat 1-4 until max number of trials is reached.

---

The advantages and necessity of choosing cuckoo hashing lies in:

- (a) Compressing the evaluation domain of DPF significantly. Considering items of length  $\rho$ , the original approach to construct a point function is using the keyword directly as the point  $\alpha$ , then the evaluation domain of DPF will be  $2^\rho$ . The computational complexity of *EvalAll* is  $O(\rho 2^\rho)$ , which is beyond the tolerance of some practical applications. With cuckoo hashing, we can achieve a similar level of complexity with a normal index PIR.
- (b) Allocating a unique index for each item inserted. The compressibility of the evaluation domain by the cuckoo hashing, though notable, is not a unique feature of the algorithm, as other hash functions can yield comparable outcomes. Unlike many collision resolution algorithms, the cuckoo hashing has the capability of assigning a unique index to every individual data entry, and this is necessary for our scheme. If multiple data items occupy the same index, ensuring the correctness of *EvalAll* is a challenging task.

3) *Shuffling*: The shuffling and PSI in our protocols specifically address the new problem introduced by cuckoo hashing. Multiple queries per keyword lead to multiple results, yet contain at most one valid response (or none). Therefore, to hide the additional results from the client, we introduce the shuffling and PSI. When the client learns  $DB[i]$  in the secret retrieval procedure, it also learns that which mask code  $m_i$  is applied to the queried value, resulting in the leakage of the position occupied by  $x$ , that is,  $h_1(x), h_2(x)$  or  $h_3(x)$ . The placement of  $y$  in the cuckoo table depends indirectly on all the items in the database. The leakage is subtle, yet still needs to be fixed.

To mitigate this leakage, we employ a shuffling procedure to hide the location of an item in the cuckoo table. We begin with a basic scenario that involves two hash functions  $h_1$  and  $h_2$ , and the client has an item  $x$ . This extends naturally to the case of  $p = 3$ . The client generates and sends two PIR queries tar-

getting  $h_1(x)$  and  $h_2(x)$  within the cuckoo table. Additionally, the client transmits two corresponding random mask codes,  $m_1$  and  $m_2$ , to server A. Server A chooses a random permutation  $\sigma : \{1, 2\} \rightarrow \{1, 2\}$  and masks the *EvalAll* results with shuffled mask codes, i.e.,  $s_0^b \oplus m_{\sigma(b)}$  ( $b \in \{1, 2\}$ ). Then what server B obtains after *EvalAll* is the masked PIR output, that is,  $(k_{h_1(x)}, v_{h_1(x)}) \oplus m_{\sigma(1)}, (k_{h_2(x)}, v_{h_2(x)}) \oplus m_{\sigma(2)}$ . If  $x \in DB$ , then server B must hold either  $x \oplus m_1$  or  $x \oplus m_2$ . Therefore, a two-party PSI+AD protocol can be performed to obtain the query result. The two sets from both parties are, respectively  $\{x \oplus m_1, x \oplus m_2\}$  from the client and the masked *EvalAll* results  $\{(k_{h_1(x)}, v_{h_1(x)}) \oplus m_{\sigma(1)}, (k_{h_2(x)}, v_{h_2(x)}) \oplus m_{\sigma(2)}\}$  from server B. When the client receives the intersection, the corresponding mask code is revealed to the client. Then the client is able to unmask the payload value, and the functionality of KPIR is fulfilled.

In our proposed scheme, the size of the sets provided by both input parties in the PSI protocol is relatively small. Therefore, the complexity of the PSI protocol with respect to sets is not a critical factor. We choose the KKRT16 [24] protocol, because it can be naturally transformed into a PSI+AD protocol. The specific modification process is omitted here.

The shuffling procedure prevents the client from learning the mapping between an item and its respective location within the cuckoo table, while retaining the ability to retrieve query results. With the PSI+AD protocol, the client secretly retrieves the query result. Therefore, a keyword PIR protocol is constructed and the flow chart is presented in Fig. 2. With the aid of cuckoo hashing and PSI, the DPF-PIR scheme is proposed to reduce computational and communication costs.

**Remark 1.** The random masks may introduce a false positive, where  $T[j] \neq x$  but  $T[j] \oplus m = x \oplus m'$  (for some masks  $m$  and  $m'$ ), leading to a PSI match. However, in practice, items are long enough, for example, 128 bits, so the false positive probability taken over all client items is acceptable, i.e.,  $2^{-128}$ .

### B. Correctness

The secret shares obtained by both servers satisfy the following equation,

$$\begin{aligned} s_0^i + s_1^i &= \sum_{j=0}^M (EVAL(k_0^i, j) \cdot T[j]) + \sum_{j=0}^M (EVAL(k_1^i, j) \cdot T[j]) \\ &= 0 + 0 + \dots + T[i] + \dots + 0 \\ &= T[i] \end{aligned} \tag{7}$$

We obtain the desired cuckoo table entry from full domain evaluation of both servers, and the secret retrieval happens on server B, which executes a PSI+AD protocol with the client to find the true answer. We can easily see that if the keyword  $x$  is in the database, it has to occupy a position of the queried cuckoo table. Therefore, the intersection of server B's set  $\{key_i + m_i\}$  and the client's set  $\{x + m_i\}$  definitely exists. The correctness of this protocol is self-evident.

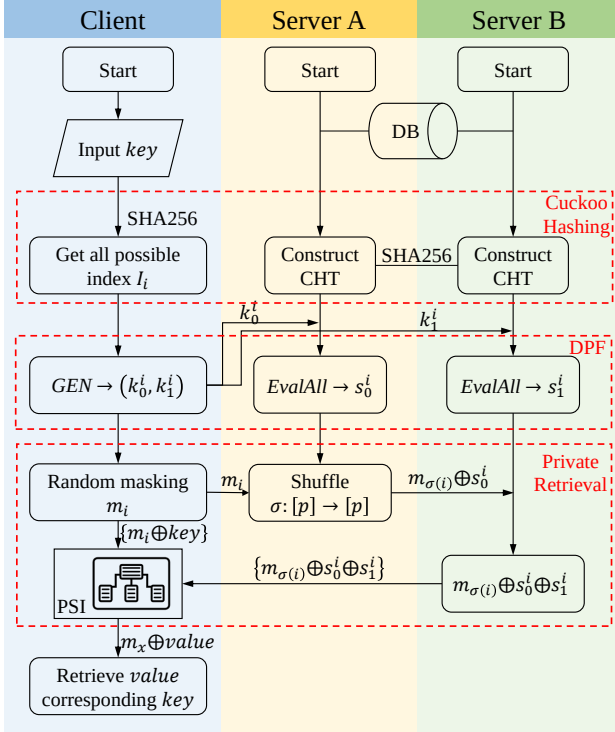


Fig. 2: The flow chart of DPF-PIR, consisting of three parts, cuckoo hashing, DPF *EvalAll* and private retrieval, as discussed in subsection III-A. The server-side hashing is precomputed, and the other two parts are performed online. CHT is an abbreviation of cuckoo hash table.

### C. Programmable DPF Variant

PDPF introduces several novel features that are practically valuable for our PIR scheme. For example, an “online” key and an “offline” key indicate the PDPF does not require two online synchronized servers. The online communication only involves the client and the online server. And the 1.5-server feature renders the non-collusion assumption more realistic. As for our keyword PIR scheme, the introduction of the online/offline feature by the PDPF causes a distinction between the relatively equivalent two servers, and the message flow of our protocol is correspondingly modified. Therefore, the protocol is divided into the online phase and offline phase, as presented in Fig. 3. The offline phase includes the offline key generation, the offline evaluation performed by the offline server, and the shuffle procedure. When the online phase begins, the client generates the online key from the offline key. Then the client sends the online key and the shuffled offline secret shares to the online server. After the online evaluation, the online server obtains shuffled and masked query result. With a final PSI+AD, the client retrieves the query result securely. In general, this PDPF-PIR variant has single online round.

In our scheme, the PDPF payload  $\beta$  is always 1, the same as the DPF usage above, to guarantee the correctness of our protocol. Thus, the big payload optimization for PDPF is not required. The privacy amplification technique based on a locally decodable code is still necessary. The complete technical description and analysis can be found in [12].

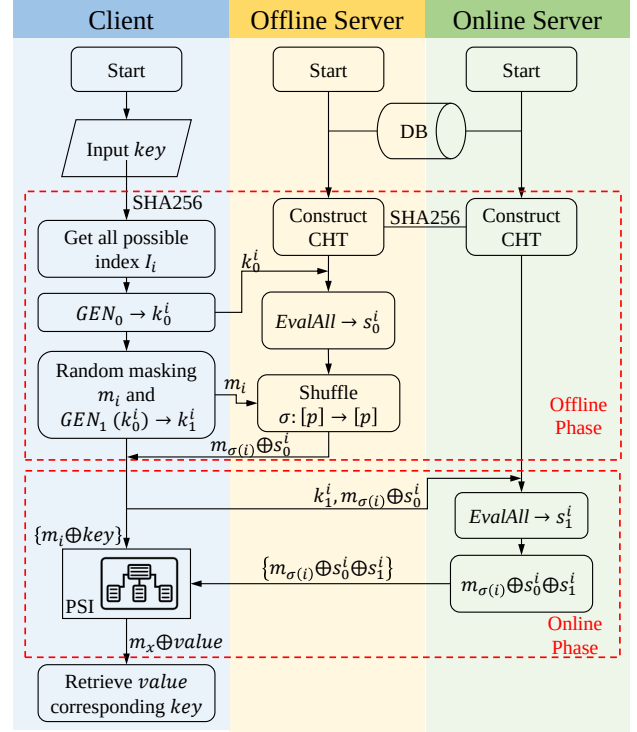


Fig. 3: The flow chart of PDPF-PIR, consisting of offline phase and online phase. The computation and communication of the offline server occur during the offline phase. Unlike DPF-PIR, only one server is engaged in the online phase.

**Remark 2.** Obviously, the PDPF has its trade-offs compared to the standard 2-party DPF. First, it cannot offer a negligible privacy error  $\epsilon$  with good concrete efficiency. Second, the running time of *GEN* and (single-point) *EVAL* scale linearly (rather than logarithmically) with the domain size.

## IV. SECURITY

In this section, we discuss the adversary model and give a formal simulation-based security proof of our proposed protocols.

### A. Adversary Model

**Semi-honest Adversary.** We consider a semi-honest adversary who controls one of the parties and follows the protocol specification exactly in the case of an honest majority. Such an adversary may try to learn more information than allowed by looking at the transcript of received messages and its internal state. Moreover, only one party is controlled by the adversary, implying the *non-collusion* assumption between the two database servers, as shown below.

**Non-colluding Servers.** If the two servers collude, they learn both DPF keys for each cuckoo table query, and learn the locations of all client’s queries into the cuckoo table. Though the hashing serves as a one-way function (OWF) from the keyword to the queried locations, these locations can leak some information about the keyword, and give the adversary an approximate guess about the keyword. Therefore, such collusion between two servers is unacceptable in our protocol.

## B. Security Proof

In this part, we provide the simulation-based security proof as follows. The functionality of keyword PIR is deterministic, so we focus on requirement of privacy.

**Theorem 1.** *If  $\{h_i | i \in [p]\}$  is a collision resistant hash function family, the DPF-PIR protocol  $\pi$  (Fig. 1) securely computes keyword PIR  $\mathcal{F}_{KPIR}$  in the presence of a semi-honest adversary who corrupts at most one party.*

**Security against Corrupted  $\mathcal{C}$ .** We construct the simulator  $\mathcal{S}_1$  as follows. It is given as input  $\mathcal{C}$ 's query keyword  $x$  and the query result  $(k, v)$ .  $\mathcal{S}_1$  runs the honest  $\mathcal{C}$  protocol with the following exceptions: For the PSI subroutine,  $\mathcal{S}_1$  honestly computes its set and runs a PSI simulator to produce a view. We prove  $\mathcal{S}_1(x, (k, v)) \stackrel{c}{\approx} \text{view}_1^\pi(x, \text{DB})$  via the following hybrid argument:

Hyb<sub>0</sub>  $\mathcal{C}$ 's view in the real protocol.

Hyb<sub>1</sub> The only difference from Hyb<sub>0</sub> is that  $\mathcal{S}_1$  constructs its input set and runs the PSI simulator to simulate the view of  $\mathcal{C}$ . This hybrid is indistinguishable from Hyb<sub>0</sub> by security of the PSI protocol. It is the simulated view of  $\mathcal{S}_1$ .

**Security against Corrupted Server A.** We construct  $\mathcal{S}_2$  as follows. It is given the database as input.  $\mathcal{S}_2$  runs the honest server A protocol with the following exceptions:  $\mathcal{S}_2$  randomly chooses mask codes  $\{m_1, \dots, m_p\}$ . For the DPF query keys,  $\mathcal{S}_2$  randomly samples a keyword  $X$  to generate simulated query keys. We prove  $\mathcal{S}_2(\text{DB}) \stackrel{c}{\approx} \text{view}_2^\pi(x, \text{DB})$  via the following hybrid argument:

Hyb<sub>0</sub> Server A's view in the real protocol.

Hyb<sub>1</sub> Same as Hyb<sub>0</sub> except that  $\mathcal{S}_2$  chooses  $p$  indexes  $\{I_1, \dots, I_p\}$  and runs *DPF.GEN* to generate simulated keys. This hybrid is computationally indistinguishable from Hyb<sub>0</sub> by security of DPF algorithm.

Hyb<sub>2</sub> Same as Hyb<sub>1</sub> except that  $\mathcal{S}_2$  randomly samples a keyword  $X \xleftarrow{\$} \{0, 1\}^*$  and computes corresponding query indexes  $\{h_i(X) | i \in [p]\}$ . This hybrid is identical to Hyb<sub>1</sub>.

Hyb<sub>3</sub> Same as Hyb<sub>2</sub> but the protocol aborts if there exists  $x, y \in \{0, 1\}^*$ ,  $x \neq y$  such that  $h_i(x) = h_i(y)$ ,  $i \in [p]$ . The aborting probability is negligible because  $h_i$  is collision resistant.

Hyb<sub>4</sub> Same as Hyb<sub>3</sub> except that  $\mathcal{S}_2$  randomly samples mask codes  $\{m_1, \dots, m_p\}$ . This hybrid is statistically identical to Hyb<sub>3</sub>.

Hyb<sub>5</sub> Same as Hyb<sub>4</sub> except that the protocol does not abort. The indistinguishability of Hyb<sub>5</sub> and Hyb<sub>4</sub> follows from the collision resistance of  $h_i$  by the same arguments as above. This hybrid is the simulated view of  $\mathcal{S}_2$ .

**Security against Corrupted Server B.** The simulator  $\mathcal{S}_3$  works mostly the same as  $\mathcal{S}_2$ . The query keys are generated in the same way. Masked secret shares  $m_{\theta(i)} \oplus s_0^i$  are also uniformly distributed. The PSI subroutine simulation is similar with  $\mathcal{S}_1$ . Therefore, we can conclude  $\mathcal{S}_3(\text{DB}) \stackrel{c}{\approx} \text{view}_3^\pi(x, \text{DB})$ .

**Remark 3.** *The security proof of PDPF-PIR is omitted here due to its similarity with Theorem 1.*

## V. IMPLEMENTATION

In this section, we implement a prototype of our keyword PIR protocols described in Fig. 2 and Fig. 3. Our prototype system is implemented using the C++ programming language and some open-source library of underlying cryptologic primitives.

### A. AES-based PRG

Since the security of the DPF construction relies on the underlying one-way function [11], that is, the PRG used to expand randomness. Hence, it is essential for the one-way function to be both difficult to exploit and efficient. The AES is a suitable algorithm for that purpose, and the leverage of AES as a PRG is widely applied in various privacy protocols. Hence, we do not elaborate on the details here. By utilizing the hardware acceleration feature of AES-NI, the OWF can maintain a high level of efficiency while leveraging the security of AES encryption. Additionally, AES-NI can provide resistance against some side-channel attacks. For the security of the DPF, the key used for AES encryption is crucial. If the key is leaked, an adversary keeping track of the communication between the client and servers is capable to break the protocol and learn all locations that client queries. Hence, it is worth paying attention to the timely replacement of keys and secure key exchange.

We can examine the computational complexity of the full domain evaluation by considering the number of PRG invocations because other operations in the full domain evaluation have a complexity of  $O(1)$ . Furthermore, the full domain evaluation is the main bottleneck of our protocols, therefore, its complexity can largely reflect the complexity of our protocols. In subsequent discussions, most references to complexity specifically denote the complexity of full domain evaluation.

### B. Cuckoo Hashing Without Stash

As stated above, the cuckoo table with a stash is undesirable in our keyword PIR constructions. If the item client queries is in the stash, there is no straightforward method to include the item in the full domain evaluation procedure and hence respond a correct response. For that purpose, we need to choose the parameters of the cuckoo table to make the possibility of insertion failure sufficiently low, i.e.,  $2^{-20}$ . According to the empirical formula in [13], we attempt to achieve a balance between the failure probability and table size parameters to identify a more suitable configuration for our protocols. Additionally, in the implementation of our protocol, we need a shared source of randomness between two servers to obtain two identical cuckoo tables with all DB entries inserted into the cuckoo table. Therefore, a Diffie-Hellman key exchange procedure is needed to generate a shared pseudorandom seed before initializing the cuckoo table.

### C. Optimized DPF Employment

First, we review the binary tree construction of DPF. The *EVAL* algorithm can be viewed as an evaluation of a leaf node, and full domain evaluation (*EvalAll*) corresponds to computing

this entire tree to learn all the values on the leaves (i.e., the whole input domain of the point function  $\hat{f}_{\alpha,\beta}$ ).

DPF full domain evaluation is the major computational overhead for the servers in our protocol. Performing a complete traversal of the database for evaluation incurs significant computational overhead. Additionally, the operation of reading the database into memory also entails substantial I/O costs.

Considering computational complexity, let  $n$  denote the number of bits in the input domain of the point function. Therefore, the size of the input domain is  $2^n$ , which can also be interpreted as the size of the database  $N$ . Thus, since the number of PRG invocations in a single *EVAL* operation is linearly related to  $n$ , the computational complexity of the regular full domain evaluation algorithm is  $O(nN)$  as a result. However, by incorporating two optimization techniques proposed in [11], the complexity is significantly reduced to  $O(N/\lceil \log(l/z) \rceil)$ , where  $l$  denotes the bits used for the representation of  $\beta$  in the base *GEN* algorithm and  $z$  denotes the bits actually required for the representation of  $\beta$ .

Compared to the original approach of constructing a DPF query with the keyword itself, Cuckoo hashing has achieved exponential speedup. When the keywords are uniformly distributed, the baseline *EvalAll* exhibits worst-case complexity of  $O(N2^{p-n})$ . Despite requiring  $p-1$  additional queries (where  $p$  denotes the number of hash functions and constructed queries per keyword), the speedup is still significant.

Furthermore, to adapt to the optimized scenario where a full domain evaluation obtains the DPF result for the entire input domain at once, a vector inner product is performed between the database and the DPF result to obtain the secret share. This replaces the scattered access to each entry in the database with a one-time access, which theoretically optimizes the I/O performance of the database stored on disk.

#### D. Extension to PDPF

The structure of PDPF-PIR is shown in Fig. 3. The most block is the same as the companion in DPF-PIR. Besides, we utilize a open-source library [25] for the underlying PPRF implementation. Lazy *GEN* computation [12] is adopted to optimize the computation of the online key, at the expense of an additional  $2^{-(N+1)}$  failure rate. The construction of PDPF [12] realizes *EvalAll* at essentially the same cost as *EVAL*, therefore, the complexity of *EvalAll* is  $O(N)$ .

#### E. Other Future Optimizations

Some more optimizations can be employed to further enhance the performance of our proposed protocol.

1) *Binning Optimization*: The main bottleneck of our PIR scheme is the computational cost of servers in computing secret shares of the query result. In this part, the computational cost of DPF *EvalAll* scales linearly with the size of the hash table. We could introduce a binning technique to decrease the size of the hash table to be queried, and it would be a significant improvement to our PIR scheme. However, after partitioning, the client reveals to the server the bins that it is querying, thereby allowing a semi-honest server to gain extra information about the client's queries. This issue can be

addressed by adding dummy queries to obfuscate the client's actual querying behavior.

2) *Amortized Query*: Answering a PIR query requires a computation linear with the size of the database. Therefore, a promising way to save computational resources is for the server to amortize costs by processing a batch of queries. At a fundamental level, amortizing PIR queries represents a strategy to reduce the aggregate number of database scans, which is especially effective when dealing with large-scale databases that are too voluminous to be processed entirely in memory.

Furthermore, certain PIR schemes [26], [27] leverage batch codes and other techniques to handle multiple query requests simultaneously. Preprocessing the database to reduce the cost of future queries, which has been proposed in several PIR schemes [2], [28], has led to significant throughput improvement.

3) *Hardware Acceleration*: SIMD (Single Instruction, Multiple Data) [29], [30] is a hardware acceleration technology that allows one instruction to perform computing on multiple pieces of data at the same time. Hence, SIMD features can be utilized in the *EvalAll* phase, where the server needs to traverse the binary tree to the leaf nodes iteratively. In our protocol, the serial part can traverse the first  $q$  levels of the tree to obtain the corresponding data of the  $2^q$  subtrees, and then SIMD can be leveraged to traverse multiple subtrees simultaneously. The parallel computation using SIMD feature can significantly enhance the server's computation efficiency and data throughput.

After the *EvalAll*, another hardware acceleration feature can be employed for optimization. AVX (Advanced Vector Extensions) is a set of instructions for x86 processors that can accelerate the performance of compute-intensive workloads [31], [32]. One such workload is the calculation of inner products. In our implementation, *EvalAll* of DPF outputs a bit vector with  $M$  elements. By an inner product between this vector and the cuckoo table vector, the secret share of each server is obtained. Thus, the AVX feature can be utilized to accelerate the computation here.

## VI. EVALUATION

Our protocol comprises two distinct operational phases: initialization phase during which the servers construct the cuckoo hash table by inserting the entire database; and query phase during which the client and servers collaboratively execute the PIR protocol. In the following analysis, we focus on the query phase because the init phase occurs only once and ahead of all the queries, which does not significantly affect the efficiency of clients' queries.

We run all experiments using a single thread of execution on an Optiplex Dell PC Desktop 12th Gen Intel(R) Core(TM) i5 12400@2.50 GHz 2.50GHz, RAM 16.00GB. We randomly choose five keywords (or indexes for standard PIR protocols) and measure the average communication and computational costs. For those protocols with preprocessing, it is challenging to establish a universally accepted evaluation methodology to measure the performance due to its two-round communication model. In this paper, we ignore the time for preprocessing



but consider the communication cost of preprocessing. Because from the client's perspective, the preprocessing time is effectively seamless, whereas the storage and communication costs remain non-negligible. Furthermore, the client may not often access the PIR service, so the preprocessing "hint" is considered as one-time.

#### A. Performance Evaluation

TABLE I: Query time and communication costs of our two schemes for a DB with a fixed size, i.e., 1GB, while the item length  $\rho$  varies. Query times are in seconds and communication costs are in Megabytes. The results are an average of 5 runs.

| Scheme   | $\log_2(N)$ | $\rho$ | Query time(s) | Communication(MB) |
|----------|-------------|--------|---------------|-------------------|
| DPF-PIR  | 30          | 1B     | 0.79          | 2.53              |
|          | 28          | 4B     | 0.41          | 2.31              |
|          | 26          | 16B    | 0.19          | 2.16              |
| PDPF-PIR | 30          | 1B     | 9.01          | 2.04              |
|          | 28          | 4B     | 4.67          | 1.58              |
|          | 26          | 16B    | 2.45          | 1.14              |

**Varying Item Length Analysis** To show the features of our schemes, we conduct experiments on our two schemes, DPF-PIR and PDPF-PIR, under varying item lengths, on an 1GB-sized DB. The results are presented in Table I. From the table, we can see that both the query time and communication costs of our schemes increase as the item length decreases, but at different rates. The growth rate of the query time roughly follows a linear pattern with respect to  $N$ , while the communication grows more slowly. This is because the growth of communication mainly stems from the extension of the DPF/PDPF key, and the length of the key follows a linear relationship with the logarithm of  $N$ .

**Time Cost Analysis** To dig deeper into our schemes, we record and illustrate the time costs of several major procedures in our schemes, i.e., cuckoo table initialization, DPF *EvalAll*, inner product and PSI+AD. The pie charts (see Fig. 4) show the proportion that these major procedures account for respectively. When the DB item length is fixed, as the DB size grows, the proportions of each segment in the pie charts (see Fig. 4 (a) for DPF-PIR and Fig. 4 (c) for PDPF-PIR) remain relatively stable. This stability arises due to the linear increase in computation for each major part, with the exception of PSI+AD. When the DB size remains constant but the item length varies, the stability of the pie charts (see Fig. 4 (b) for DPF-PIR and Fig. 4 (d) for PDPF-PIR) is lost. As the item length decreases gradually and the number of items increases accordingly, the proportions for *EvalAll* and cuckoo table initialization escalate. The reason is that, in this scenario, the time consumed by the inner product procedure remains roughly constant, while the computational load of *EvalAll* and cuckoo table initialization increases linearly with the number of items. When we compare the two schemes vertically, i.e., Fig. 4 (a) and Fig. 4 (c), it is observed that PDPF-PIR consumes more time on *EvalAll* than DPF-PIR, which is attributed to the more complex primitives PDPF employs.

The pie charts also illustrate the performance bottleneck of our schemes. Further optimization of *EvalAll* and the inner product could enhance the efficiency of our schemes, which guides our future work.

**Performance Comparison** Table II provides a detailed comparison of the performance characteristics across different schemes. We selected representative schemes from the table to conduct a performance comparison with our own schemes, as shown in Table III.

TABLE II: A comparison of PIR schemes on database size  $N$  and security parameter  $\lambda$ . The overhead column indicates whether the server computation per database bit is at most polylogarithmic in  $\lambda$ .

| Scheme          | Servers | Communication | Pre-processing | Polylog( $\lambda$ ) compute overhead |
|-----------------|---------|---------------|----------------|---------------------------------------|
| <b>DPF-PIR</b>  | 2       | $\log N$      | $\times$       | $\checkmark$                          |
| <b>PDPF-PIR</b> | 2       | $\log N$      | $\times$       | $\checkmark$                          |
| XPIR [33]       | 1       | $\sqrt{N}$    | $\checkmark$   | $\checkmark$                          |
| SealPIR [26]    | 1       | $\sqrt{N}$    | $\checkmark$   | $\checkmark$                          |
| FastPIR [3]     | 1       | $N$           | $\checkmark$   | $\checkmark$                          |
| OnionPIR [4]    | 1       | $\log N$      | $\checkmark$   | $\checkmark$                          |
| Spiral [34]     | 1       | $\sqrt{N}$    | $\times$       | $\checkmark$                          |
| SimplePIR [2]   | 1       | $\sqrt{N}$    | $\checkmark$   | $\checkmark$                          |

TABLE III: Query time and communication costs of our two schemes and other representative PIR schemes under varying item number  $N$ , while the item length  $\rho$  is fixed respectively. Query times are in seconds and communication costs are in Megabytes. The throughput is computed based on the query time. The results are an average of 5 runs.

| DB size | Scheme        | $\log_2(N)$ | $\rho$ | Query time(s) | Communication(MB) | Throughput(MB/s) |
|---------|---------------|-------------|--------|---------------|-------------------|------------------|
| 1GB     | DPF-PIR       | 26          | 16B    | 0.19          | 2.16              | 5481             |
|         | PDPF-PIR      | 26          | 16B    | 2.45          | 1.14              | 418              |
|         | XOR PIR       | 33          | 1b     | 0.17          | 1024              | 6067             |
|         | SealPIR [26]  | 22          | 288B   | 11.88         | 5.27              | 97               |
|         | XPIR [33]     | 20          | 1KB    | 7.21          | 16.21             | 142              |
|         | FastPIR [3]   | 20          | 1KB    | 4.76          | 32.35             | 215              |
|         | SimplePIR [2] | 20          | 1KB    | 0.10          | 121.24            | 10305            |
|         |               |             |        |               |                   |                  |
| 2GB     | DPF-PIR       | 27          | 16B    | 0.45          | 2.31              | 4551             |
|         | PDPF-PIR      | 27          | 16B    | 5.24          | 1.58              | 391              |
|         | XOR PIR       | 34          | 1b     | 0.39          | 2048              | 5251             |
|         | SealPIR       | 23          | 288B   | 25.33         | 9.76              | 91               |
|         | XPIR          | 21          | 1KB    | 16.47         | 25.12             | 124              |
|         | FastPIR       | 21          | 1KB    | 9.78          | 68.43             | 209              |
|         | SimplePIR     | 21          | 1KB    | 0.23          | 232.18            | 8904             |
|         |               |             |        |               |                   |                  |
| 4GB     | DPF-PIR       | 28          | 16B    | 0.95          | 2.54              | 4312             |
|         | PDPF-PIR      | 28          | 16B    | 11.45         | 2.06              | 358              |
|         | XOR PIR       | 35          | 1b     | 0.87          | 4096              | 4708             |
|         | SealPIR       | 24          | 288B   | 54.78         | 15.34             | 84               |
|         | XPIR          | 22          | 1KB    | 35.32         | 55.46             | 116              |
|         | FastPIR       | 22          | 1KB    | 21.27         | 135.43            | 193              |
|         | SimplePIR     | 22          | 1KB    | 0.49          | 467.87            | 8359             |
|         |               |             |        |               |                   |                  |

All the experiments were carried out without considering the network conditions, i.e., all the computations are carried out by a single process.

We first measure the throughput of each PIR scheme on a database roughly 1 GB in size, where we take the item length to be that for which the highest throughput is reported in the corresponding paper (or in a related paper, if it is not made explicit). Then we conduct experiments on these schemes with the corresponding item length under different DB size settings, i.e., 1GB, 2GB, 4GB. These item length settings and performance results are presented in Table III.

XOR PIR scheme is the most basic multi-server scheme, and we benchmark this scheme to provide a hard upper bound

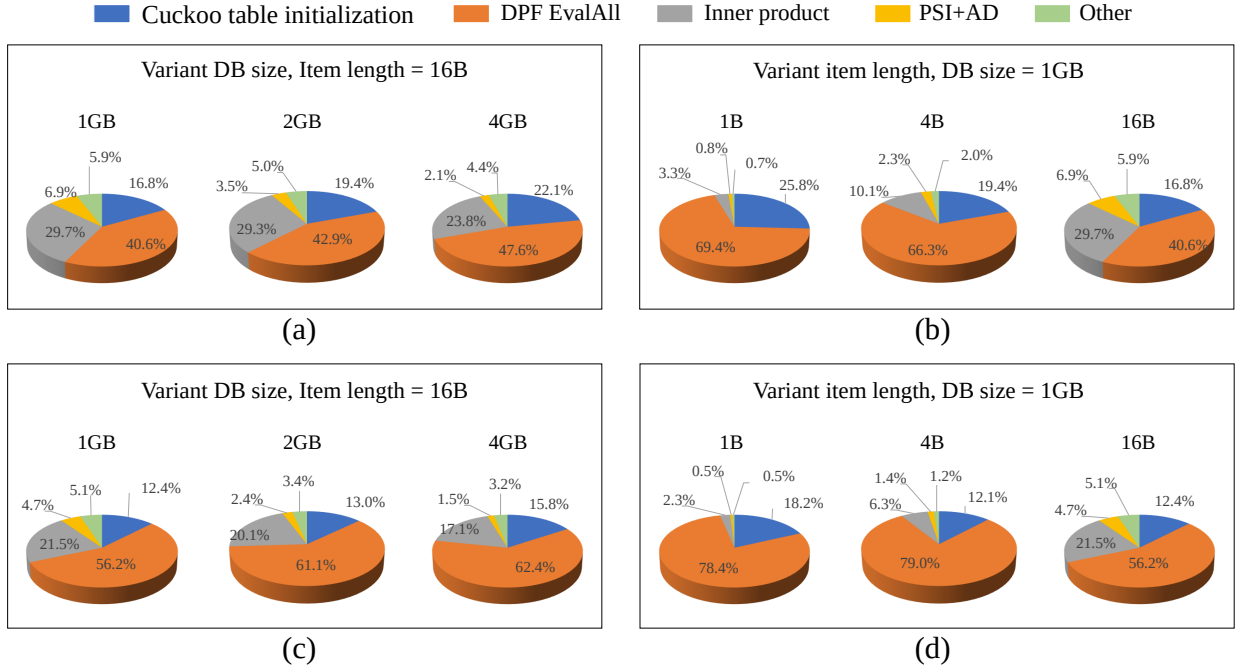


Fig. 4: Time costs of different procedures in our proposed keyword PIR schemes under different DB settings. (a) DPF-PIR with varying DB size, i.e., 1GB, 2GB 4GB. (b) DPF-PIR with varying DB item length, i.e., 1B, 4B, 16B. (c) PDPF-PIR with varying DB size, i.e., 1GB, 2GB 4GB. (d) PDPF-PIR with varying DB item length, i.e., 1B, 4B, 16B.

on the speed of the two-server PIR, which is 6.1GB/s. Our DPF-PIR protocol achieves throughput of 5.5GB/s, which is significantly faster than some single-server PIR schemes, 25x faster than FastPIR. As for PDPF-PIR, due to the increased computational complexity brought by the PPRF primitive, the throughput decreases on a significant scale. However, it is still faster than most of PIR schemes (like FastPIR) according to the results shown in Fig. 5. The outlier of our experiment results is SimplePIR, and it achieves the fastest throughput in our experiment setting, 10GB/s. Benefitting from the well-designed data structure and preprocessing technique, in the query phase of SimplePIR, the server does not need to compute heavy cryptographic operations, which gives it great efficiency. Nonetheless, our throughput is normalized by dividing the total throughput by the number of servers, that is, for each server, the computation burden of our protocol and SimplePIR is on the same level. Moreover, our protocol has an advantage in the communication cost, as presented below.

As for the communication cost, due to the compressed input space and sublinear key length, the communication cost of DPF is very low, which makes our protocol greatly communication-efficient. Compared with other schemes, our protocol achieves the lowest communication burden as shown in Fig. 6. Despite the performance in terms of the throughput costs, PDPF has an advantage in the communication aspect. With an offline key, PDPF only need one key instead of two keys in the DPF setting to be transmitted to the server side. Therefore, the communication cost of the PDPF is roughly half the DPF, and the data in Table III also shows the same results. Moreover, when the DB size varies, the throughput

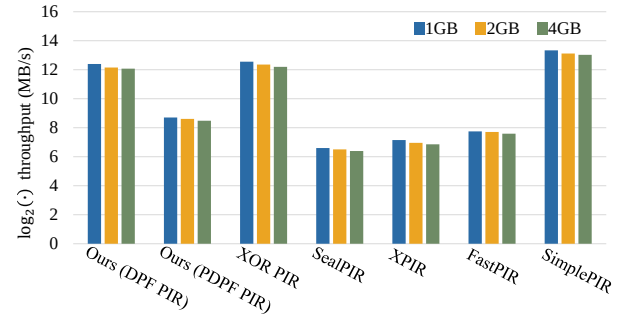


Fig. 5: Throughput results for different PIR protocols. The value is normalized by  $\log_2(\cdot)$ .

and communication results also change accordingly. However, overall performance features of each scheme are preserved, as presented in Fig. 5 and Fig. 6.

## VII. RELATED WORKS

**Verifiable PIR.** Verifiable PIR (VPIR) allows a client to perform PIR on data, while enabling the client to verify the correctness of the retrieved data. VPIR schemes typically employ encryption, hashing and other techniques to ensure data privacy and integrity, enabling the client to obtain the correct query results even if some servers are in a malicious state. A recent work [35] in this line combines a linear map commitment scheme with a linear PIR scheme to achieve a VPIR scheme, which guarantees that the client can still obtain the correct information even when all servers are malicious.

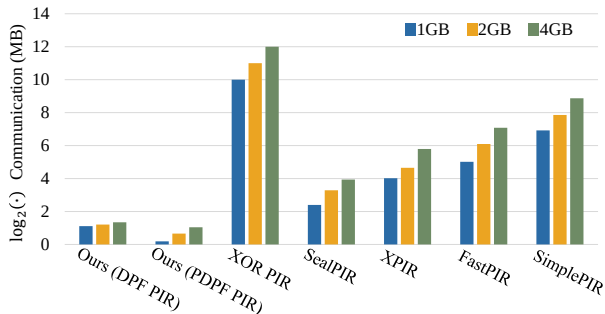


Fig. 6: Communication results for different PIR protocols. The value is normalised by  $\log_2(\cdot)$ .

The discussion on the verifiable DPF commences in [11]. A lightweight verifiable two-server DPF is introduced in [36], followed by a verifiable multi-server DPF in [37]. The leverage of verifiable DPF in our schemes is natural, and improves the security against malicious adversaries in practice.

**Fault Tolerant PIR.** In a fault-tolerant PIR scheme, the system is designed to handle various types of faults that may occur during the PIR process. These faults could include server failures, network failures, or even malicious attacks on the system. The goal is to ensure that the PIR scheme can continue to operate correctly and securely despite these potential issues. This was first studied in [38], and the Byzantine setting is considered in [39], [40]. These works are designed in the replicated database setting. A threshold version of DPF for PIR is introduced in [41], and a recent work [42] focuses on the combination of erasure coded storage and fault-tolerant DPF to construct a state-of-the-art fault-tolerant PIR protocol. Nevertheless, these research focus on the multi-server (more than two). Fault tolerance in the two-server setting is an interesting topic, and more efforts are needed to address it. In this paper, we focus on the performance of PIR protocols, and cuckoo hashing is presented to reduce the communication cost and improve computation.

**IT-PIR** [6], [42], [43]. A trivial solution to the PIR problem is to send the entire database to the user. However, while being perfectly private, the communication complexity of this solution may be prohibitively large. If the server is not allowed to gain any information about the identity of the retrieved data, the linear communication complexity of the trivial solution is optimal [6]. To solve this problem, Chor et al. [6] suggested that the user access  $k$  replicated copies of the database stored at different servers, such that each individual server get absolutely no information about  $i$ . PIR in such setting is called information-theoretic PIR (IT-PIR). The leverage of DPF in a IT-PIR protocol provides a efficient way to compress the query size, which is shown in our evaluation section.

**cPIR** [2], [26], [34]. Unlike IT-PIR, cPIR relies on computational complexity assumptions such as public-key cryptography, homomorphic encryption, and other cryptographic primitives for its security. Many recent works employ a preprocessing phase ahead of the client’s query to enhance the overall protocol throughput to achieve a sublinear response

time. Ring-LWE based XPIR [33] pre-computes the data importation into NTT-CRT form polynomials and computes the query result with polynomials. In XPIR, a query comprises a vector of ciphertexts, with its size matching the number of elements in the database. In contrast, SealPIR [26] involves the client sending a single ciphertext containing an encoding of the desired element’s index and employs an oblivious expansion procedure to extract the corresponding ciphertext vector from this single ciphertext. This optimization significantly reduces the computation and communication costs of a client’s query, but brings more computation burden to the server. The idea of the expansion procedure shares some similarity with the role played by DPF in our protocols. FastPIR [3] further enhances efficiency by performing computations on compressed PIR queries, thus eliminating the time needed to uncompress queries. It also employs a similar approach to compress the responses, compensating for the expansion caused by computation on compressed queries. In contrast to the aforementioned schemes, SimplePIR [2], based on LWE, offers a simplified implementation and computation without involving ciphertext compression and expansion procedures. Additionally, SimplePIR also employs a preprocessing phase, which generates a “hint” of the database to enhance the throughput of the query phase. While the online query phase remains linear in the size of the database rather than sublinear, as in the above schemes, it nevertheless proves to be concretely efficient, resulting in a significantly larger throughput, as shown in Table III in Section VI.

**Prior use of DPFs in systems** [42], [44]. Splinter [45] uses function secret sharing (both DPFs and range queries) to allow users to efficiently make private queries on a public and immutable database. DURASIFT [46] uses DPFs with MPC across multiple servers to support boolean expressions of keyword searches for multiple users without leaking search access patterns. However, its techniques incur significant overhead in comparison to ours, and the authors consider thousands rather than millions of documents. Floram [15] uses DPFs to implement a distributed-trust ORAM that has linear costs but fast concrete performance. Metadata-hiding communication also benefits from DPFs (e.g. Riposte [47] and Express [44]). Some of these works utilize a PIR protocol based on DPF as a building block. Our KPIR protocols can also serve as a lower layer of other security applications.

## VIII. CONCLUSION

In this paper, we have presented a two-server keyword PIR protocol, DPF-PIR and a variant with online/offline extension, PDPF-PIR. To improve the performance of our protocols, we employ various primitives while maintain privacy and security of our protocols, which has been validated through the simulation-based proof. With a comprehensive evaluation, we have shown that our KPIR scheme based on DPF has advantages over existing PIR schemes under the non-collusion assumption, that is, less computation and communication, as presented in the evaluation. Moreover, a PDPF variant also makes the assumption more acceptable in practice. The direction of future research can be the relaxation of non-collusion assumption and a further optimization, such as the

exploration of an optimal cuckoo hashing parameters and other alternative hashing schemes.

## REFERENCES

- [1] E. Boyle, N. Gilboa, and Y. Ishai, “Function Secret Sharing,” in *Advances in Cryptology - EUROCRYPT 2015*, 2015, pp. 337–367.
- [2] A. Henzinger, M. M. Hong, H. Corrigan-Gibbs, S. Meiklejohn, and V. Vaikuntanathan, “One server for the price of two: Simple and fast single-server private information retrieval,” pp. 3889–3905, 2023.
- [3] I. Ahmad, Y. Yang, D. Agrawal, A. E. Abbadi, and T. Gupta, “Addr: Metadata-private voice communication over fully untrusted infrastructure,” pp. 313–329, 2021.
- [4] M. H. Mughees, H. Chen, and L. Ren, “OnionPIR: Response Efficient Single-Server PIR,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 2292–2306.
- [5] S. M. Hafiz and R. Henry, “A Bit More Than a Bit Is More Than a Bit Better: Faster (essentially) optimal-rate many-server PIR,” *Proceedings on Privacy Enhancing Technologies*, pp. 112–131, 2019.
- [6] B. Chor, E. Kushilevitz, O. Goldreich, and M. Sudan, “Private information retrieval,” *Journal of the ACM (JACM)*, pp. 965–981, 1998.
- [7] O. Goldreich, *The Foundations of Cryptography - Volume 2: Basic Applications*, 2004.
- [8] E. Kushilevitz and R. Ostrovsky, “Replication is not needed: Single database, computationally-private information retrieval,” in *Proceedings 38th Annual Symposium on Foundations of Computer Science*, 1997, pp. 364–373.
- [9] T. Mayberry, E. Blass, and A. H. Chan, “Efficient private file retrieval by combining ORAM and PIR,” in *21st Annual Network and Distributed System Security Symposium, NDSS*, 2014.
- [10] N. Gilboa and Y. Ishai, “Distributed Point Functions and Their Applications,” in *Advances in Cryptology - EUROCRYPT 2014*, 2014, pp. 640–658.
- [11] E. Boyle, N. Gilboa, and Y. Ishai, “Function Secret Sharing: Improvements and Extensions,” in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 2016, pp. 1292–1303.
- [12] E. Boyle, N. Gilboa, Y. Ishai, and V. I. Kolobov, “Programmable Distributed Point Functions,” in *Advances in Cryptology - CRYPTO 2022*, 2022, pp. 121–151.
- [13] D. Demmler, P. Rindal, M. Rosulek, and N. Trieu, “PIR-PSI: Scaling Private Contact Discovery,” *Proceedings on Privacy Enhancing Technologies*, pp. 159–178, 2018.
- [14] E. Dauterman, E. Feng, E. Luo, R. A. Popa, and I. Stoica, “DORY: an encrypted search system with distributed trust,” Tech. Rep.
- [15] J. Doerner and A. Shelat, “Scaling ORAM for secure computation,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017*, 2017, pp. 523–535.
- [16] S. Servan-Schreiber, K. Hogan, and S. Devadas, “Adveil: A private targeted-advertising ecosystem,” p. 1032, 2021.
- [17] D. Boneh, K. Lewi, and D. J. Wu, “Constraining Pseudorandom Functions Privately,” in *Public-Key Cryptography - PKC 2017*, 2017, pp. 494–524.
- [18] D. Boneh, S. Kim, and H. Montgomery, “Private Puncturable PRFs from Standard Lattice Assumptions,” in *Advances in Cryptology - EUROCRYPT 2017*, 2017, pp. 415–445.
- [19] D. Boneh and B. Waters, “Constrained Pseudorandom Functions and Their Applications,” in *Advances in Cryptology - ASIACRYPT 2013*, 2013, pp. 280–300.
- [20] A. Kiayias, S. Papadopoulos, N. Triandopoulos, and T. Zacharias, “Delegatable pseudorandom functions and applications,” in *Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security*, 2013, pp. 669–684.
- [21] A. Kirsch, M. Mitzenmacher, and U. Wieder, “More robust hashing: Cuckoo hashing with a stash,” *SIAM Journal on Computing*, pp. 1543–1561, 2010.
- [22] X. Li, D. G. Andersen, M. Kaminsky, and M. J. Freedman, “Algorithmic improvements for fast concurrent Cuckoo hashing,” in *Proceedings of the Ninth European Conference on Computer Systems*, 2014, pp. 1–14.
- [23] B. Pinkas, T. Schneider, and M. Zohner, “Scalable Private Set Intersection Based on OT Extension,” *ACM Transactions on Privacy and Security*, pp. 1–35, 2018.
- [24] V. Kolesnikov, R. Kumaresan, M. Rosulek, and N. Trieu, “Efficient batched oblivious PRF with applications to private set intersection,” in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 2016, pp. 818–829.
- [25] bo hub, “Puncturable\_prf,” [https://github.com/bo-hub/Puncturable\\_PRf](https://github.com/bo-hub/Puncturable_PRf), 2021.
- [26] S. Angel, H. Chen, K. Laine, and S. Setty, “PIR with Compressed Queries and Amortized Query Processing,” in *2018 IEEE Symposium on Security and Privacy (SP)*, 2018, pp. 962–979.
- [27] W. Lueks and I. Goldberg, “Sublinear Scaling for Multi-Client Private Information Retrieval,” in *Financial Cryptography and Data Security*, 2015, pp. 168–186.
- [28] E. Boyle, Y. Ishai, R. Pass, and M. Wootters, “Can we access a database both locally and privately?” in *Theory of Cryptography - 15th International Conference, TCC 2017*, 2017, pp. 662–693.
- [29] J. Zhou and K. A. Ross, “Implementing database operations using SIMD instructions,” in *Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data*, 2002, pp. 145–156.
- [30] D. Nuzman, I. Rosen, and A. Zaks, “Auto-vectorization of interleaved data for SIMD,” *SIGPLAN Not.*, pp. 132–143, 2006.
- [31] H. Jeong, S. Kim, W. Lee, and S. Myung, “Performance of SSE and AVX instruction sets,” 2012.
- [32] R. Lim, Y. Lee, R. Kim, and J. Choi, “An implementation of matrix-matrix multiplication on the Intel KNL processor with AVX-512,” *Cluster Computing*, pp. 1785–1795, 2018.
- [33] C. Aguilar-Melchor, J. Barrier, L. Fousse, and M.-O. Killijian, “XPIR: Private Information Retrieval for Everyone,” *Proceedings on Privacy Enhancing Technologies*, pp. 155–174, 2016.
- [34] S. J. Menon and D. J. Wu, “SPIRAL: Fast, High-Rate Single-Server PIR via FHE Composition,” in *2022 IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 930–947.
- [35] Q. Cao, H. Tran, S. H. Dau, X. Yi, E. Viterbo, C. Feng, Y. Huang, J. Zhu, S. Kruglik, and H. M. Kiah, “Committed private information retrieval,” in *Computer Security - ESORICS 2023*, 2023, pp. 393–413.
- [36] L. de Castro and A. Polychroniadou, “Lightweight, maliciously secure verifiable function secret sharing,” in *Advances in Cryptology - EUROCRYPT 2022*, 2022, pp. 150–179.
- [37] S. Kruglik, S. H. Dau, H. M. Kiah, H. Wang, and L. F. Zhang, “Verifiable information-theoretic function secret sharing,” p. 453, 2024.
- [38] A. Beimel and Y. Stahl, “Robust information-theoretic private information retrieval,” *J. Cryptol.*, pp. 295–321, 2007.
- [39] C. Devet, I. Goldberg, and N. Heninger, “Optimally robust private information retrieval,” in *Proceedings of the 21th USENIX Security Symposium*, 2012, pp. 269–283.
- [40] I. Goldberg, “Improving the robustness of private information retrieval,” in *2007 IEEE Symposium on Security and Privacy (SP)*, 2007, pp. 131–148.
- [41] P. Bunn, E. Kushilevitz, and R. Ostrovsky, “CNF-FSS and its applications,” in *Public-Key Cryptography - PKC 2022*, 2022, pp. 283–314.
- [42] A. Park, T. Leong, F. Maturana, W. Zheng, and R. Vinayak, “Communication-efficient, fault tolerant pir over erasure coded storage,” in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024, pp. 167–167.
- [43] D. Demmler, A. Herzberg, and T. Schneider, “RAID-PIR: Practical Multi-Server PIR,” in *Proceedings of the 6th Edition of the ACM Workshop on Cloud Computing Security*, 2014, pp. 45–56.
- [44] S. Eskandarian, H. Corrigan-Gibbs, M. Zaharia, and D. Boneh, “Express: Lowering the Cost of Metadata-hiding Communication with Cryptographic Privacy,” *arXiv:1911.09215*, 2020.
- [45] F. Wang, C. Yun, S. Goldwasser, V. Vaikuntanathan, and M. Zaharia, “Splinter: Practical Private Queries on Public Data,” in *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, 2017, pp. 299–313.
- [46] B. H. Falk, S. Lu, and R. Ostrovsky, “DURASIFT: A Robust, Decentralized, Encrypted Database Supporting Private Searches with Complex Policy Controls,” in *Proceedings of the 18th ACM Workshop on Privacy in the Electronic Society*, 2019, pp. 26–36.
- [47] H. Corrigan-Gibbs, D. Boneh, and D. Mazières, “Riposte: An Anonymous Messaging System Handling Millions of Users,” in *2015 IEEE Symposium on Security and Privacy (SP)*, 2015, pp. 321–338.